Autonomous Whitepaper Production

From intent to polished artifacts via deterministic gates + Codex iteration

Adam Boas

Operator / Strategist

January 2026

Distribution: Public (LinkedIn). **Disclaimer:** Views are the author's and do not represent official policy.

Contents

1 Introduction 3
2 The Pipeline (Architecture) 3
3 Quality Gates (Deterministic) 3
4 Semantic Review (Cheap vs Full) 3
5 Iteration Loop (Patch-based) 4
6 Scalability and Governance 4
7 Risks and Mitigations 4
8 Recommendations 4
9 Conclusion 4
10 References 4

Executive Summary

This paper proposes a fully automated writing workflow that turns a simple intent specification into publish-ready strategy artifacts. The core claim is operational: quality improves when writing is treated like software—versioned, testable, iterated, and gated.

- **Problem:** Most whitepapers are slow, untestable, and depend on scarce human attention.
- **Insight:** Writing becomes scalable when we define "done" up front and enforce it with gates.
- **Approach:** Intent/spec → deterministic checks → cheap semantic feedback → full semantic review → iterative patch loop → publish.
- **Outcome:** Faster cycles, higher consistency, auditable decision trails, and parallel production across many papers.

1 Introduction

Organizations treat software with discipline and writing with vibes. The result is predictable: policy and strategy documents drift, recommendations lack owners, and revision cycles stall because the process is not instrumented.

2 The Pipeline (Architecture)

The pipeline begins with two inputs:

- **INTENT.md:** the paper's purpose, required sections, constraints, and definition of "polished."
- **PAPER.yml:** variables (title, author, date, audience, assets), enabling repeatable rendering.

The output is not "a document." The output is a set of artifacts produced by a reproducible process: PDF for humans and HTML for systems.

> **Key takeaway:** When the spec is explicit, the writing loop becomes automatable.

3 Quality Gates (Deterministic)

Deterministic gates catch structural failures before any semantic judgment is attempted:

- Required files exist (INTENT, variables).
- Required sections exist.
- No placeholders (e.g., draft markers left in the text).
- Builds succeed (PDF/HTML).

These checks are cheap, fast, and objective. They eliminate low-value review cycles.

4 Semantic Review (Cheap vs Full)

After deterministic gates pass, we layer semantic checks:

- **Cheap semantic pass:** low-cost scoring across categories (clarity, specificity, actionability).

This provides frequent feedback.

- **Full semantic pass:** an expensive review triggered only when a PR is labeled for full review. This is where deep critique happens.

5 Iteration Loop (Patch-based)

The iteration loop applies improvements as diffs. Each cycle:

1. Run checks and generate prioritized fixes.
2. Apply fixes as a constrained patch (only the paper source).
3. Re-run gates.
4. Stop when no actionable recommendations remain or the "polished" threshold is met.

6 Scalability and Governance

This approach scales because it is parallel:

- Many papers can evolve simultaneously in separate folders.
- Each paper has a clear spec and a local definition of done.
- Reviews are auditable: diffs, comments, labels, and artifacts.

7 Risks and Mitigations

- **Model hallucination:** mitigate with deterministic gates and strict patch constraints.
- **Cost:** mitigate with tiered analysis and label-triggered full review.
- **Over-automation:** keep PR labels as an escape hatch for human intervention.

8 Recommendations

1. Adopt a per-paper spec (INTENT + variables) as the entry requirement.
2. Enforce deterministic gates before semantic checks.
3. Run cheap semantic feedback continuously; reserve full reviews for labeled PRs.
4. Iterate via patch constraints until the rubric's "polished" criteria are met.

9 Conclusion

If we can automate tests for code, we can automate tests for writing. The objective is not to eliminate thinking; it is to eliminate wasted cycles. The result is faster, more consistent, more governable strategy.

10 References

- Docs-as-code practices and CI workflows for documentation.
- Version control review patterns (branches, PRs, gates, releases).